

```
exercise4.py x
1 class Median:
2
3     def median(self, arr):
4         n = len(arr)
5         arr = sorted(arr)
6         return arr[n//2] if n % 2 == 1 else (arr[n//2-1] + arr[n//2]) / 2
7
8     def calculate_median(self, x1, x2, x3=None, x4=None, x5=None):
9         arr = [x1, x2]
10        if x3 is not None:
11            arr.append(x3)
12        if x4 is not None:
13            arr.append(x4)
14        if x5 is not None:
15            arr.append(x5)
16        return self.median(arr)
17
18
19 m = Median()
20 print(m.calculate_median(3, 5, 1, 4, 2))
21 print(m.calculate_median(8, 6, 4, 2))
22 print(m.calculate_median(9, 3, 7))
23 print(m.calculate_median(5, 2))
24
```

# Polymorphism Exercise 4

## Exercise 4

Create the `Median` that has the method `calculate_median` that calculates the median of the integers passed to the method. Use method overloading so that this method can accept anywhere from two to five parameters.

► How to calculate median

| Method Call                                    | Return Value |
|------------------------------------------------|--------------|
| <code>m.calculate_median(3, 5, 1, 4, 2)</code> | 3            |
| <code>m.calculate_median(8, 6, 4, 2)</code>    | 5.0          |
| <code>m.calculate_median(9, 3, 7)</code>       | 7            |
| <code>m.calculate_median(5, 2)</code>          | 3.5          |

TRY IT

```
3
5.0
7
3.5
```

Submit your code for evaluation

Since the numbers need to be in order, the parameters are placed into a list and the list is sorted. If the list has an odd number of elements, then the middle element is returned. If the number of elements are even, the average of the two middle elements is returned. Here is one possible solution:

```
class Median:
    def calculate_median(self, n1, n2, n3=None, n4=None, n5=None):
        if n3 is not None and n4 is not None and n5 is not None:
            numbers = [n1, n2, n3, n4, n5]
        elif n3 is not None and n4 is not None:
            numbers = [n1, n2, n3, n4]
        elif n3 is not None:
            numbers = [n1, n2, n3]
        else:
            numbers = [n1, n2]

        numbers.sort()
        median_index = len(numbers) // 2
        if len(numbers) % 2 == 0:
            median = (numbers[median_index] + numbers[median_index - 1]) / 2
        else:
            median = numbers[median_index]
        return median
```

Check It!

LAST RUN on 12/28/2023, 2:53:40 AM

Test passed

Next